Panduan Neo4j
untuk Sistem Pengetahuan VSD, Motor, dan Pompa

Dokumentasi teknis yang ditulis dengan bahasa umum

Tujuan dokumen
Membantu pembaca memahami apa yang disimpan di Neo4j, bagaimana informasi saling berhubungan, apa yang boleh ditambahkan di masa depan, dan bagaimana database dijaga agar tetap benar, cepat, dan mudah dikembangkan.

Dokumen ini ditujukan untuk:

- Staff operasional dan maintenance yang ingin memahami fungsi database tanpa harus menguasai pemrograman.
- Supervisor dan engineer yang perlu melihat arah pengembangan sistem.
- Developer yang akan mengembangkan aplikasi, query, GraphRAG, atau integrasi AI.
- Orang baru di tim yang membutuhkan gambaran menyeluruh sebelum masuk ke detail teknis.

Status database saat dokumentasi ini dibuat:

Bagian	Kondisi
Struktur graph utama	Stabil dan sudah diaudit
FaultCode	195 data, lengkap
WarningEvent	178 data, lengkap
MitigationAction	373 tindakan, lengkap
Bahasa pengguna	373 dari 373 event sudah tersedia
Hubungan event -> tindakan	373 hubungan, tanpa orphan dan tanpa multi-owner

1. Neo4j itu apa?

Neo4j adalah database yang menyimpan informasi beserta hubungan antar-informasi. Kalau database tabel seperti lembar Excel cocok untuk data berbentuk baris dan kolom, Neo4j cocok ketika kita perlu memahami "apa berhubungan dengan apa".

Bayangkan seperti peta hubungan

Sebuah pompa tidak berdiri sendiri. Pompa terhubung ke motor, motor dikendalikan VSD, VSD memiliki parameter, parameter dapat membentuk indikasi, indikasi dapat mengarah ke kemungkinan yang perlu diperiksa, dan hasil pemeriksaan dapat mengarah ke tindakan.

Contoh sederhana:

VSD	**menggerakkan**	**Motor**	**menggerakkan**	**Pompa**

Fault	**memiliki tindakan**	**Mitigation Action**

Kekuatan Neo4j bukan hanya menyimpan kata "motor" atau "fault", tetapi menyimpan hubungan yang membuat informasi tersebut bisa ditelusuri.

Mengapa tidak cukup memakai tabel biasa?

Tabel biasa tetap berguna untuk telemetry, laporan, atau data transaksi. Namun ketika pertanyaannya berubah menjadi "fault ini berlaku untuk produk apa?", "tindakan apa yang terkait?", "kemungkinan apa yang perlu diperiksa?", Neo4j membuat penelusuran hubungan menjadi lebih alami.

2. Peran Neo4j dalam sistem

Dalam sistem ini, Neo4j berperan sebagai pusat pengetahuan. Aplikasi mengambil informasi dari Neo4j, kemudian menampilkannya ke pengguna dalam bentuk yang lebih mudah dibaca.

Pengguna	Aplikasi	Neo4j	Hasil terstruktur	Tampilan / AI

Neo4j bukan pengganti semua komponen. Telemetry real-time, tampilan aplikasi, machine learning, dan AI dapat berada di luar Neo4j. Neo4j menyimpan pengetahuan dan hubungan yang perlu dipahami sistem.

Prinsip penting

AI sebaiknya tidak menjadi sumber kebenaran. Neo4j menyimpan fakta dan hubungan yang sudah dikendalikan; AI membantu menjelaskan hasilnya dengan bahasa manusia.

Apa yang saat ini disimpan?

Kelompok	Isi	Bahasa sederhana
Referensi perangkat	Product, FirmwareRevision, PumpModel	Perangkat apa yang dimaksud dan konteks produknya.
Parameter & indikator	Parameter, Indicator, Formula	Data apa yang dilihat dan bagaimana suatu kondisi dihitung.
Pola & kemungkinan	EvidencePattern, ConditionHypothesis	Kondisi apa yang terlihat dan kemungkinan apa yang perlu diperiksa.
Pemeriksaan	InspectionCheck, InspectionStep	Apa yang perlu dicek dan bagaimana cara mengeceknya.
Event resmi VSD	FaultCode, WarningEvent	Kode fault atau warning dari perangkat.
Tindakan resmi	MitigationAction	Apa tindakan yang disarankan untuk event tersebut.
Sumber	Source	Dari dokumen atau referensi mana informasi berasal.

3. Cara membaca hubungan di dalam database

Istilah seperti "node" dan "relationship" sering terdengar rumit. Sebenarnya konsepnya sederhana:

Istilah	Arti sederhana	Contoh
Node	Satu benda, konsep, atau informasi.	FaultCode 2310, Parameter Motor Current, Product ACQ580.
Relationship	Hubungan antara dua node.	FaultCode HAS_ACTION MitigationAction.
Property	Keterangan yang dimiliki node.	code, name, user_description, product_name.

Cara paling mudah membaca graph

Baca dari kiri ke kanan seperti kalimat. Contoh: "FaultCode 2310 memiliki tindakan: Periksa beban, motor, dan kabel motor."

Dua jalur utama yang perlu dibedakan

A. Jalur event resmi dari VSD

Fault / Warning	->	Tindakan

Jalur ini berasal dari event perangkat. Jika VSD memberikan kode, database membantu menerjemahkan arti dan tindakan terkait.

B. Jalur penalaran pemeriksaan

Parameter	->	Indikasi	->	Pola	->	Kemungkinan	->	Pemeriksaan

Jalur ini dipakai ketika sistem melihat kondisi data dan ingin membantu teknisi menentukan apa yang perlu diperiksa. Jalur ini tidak boleh dipahami sebagai diagnosis otomatis.

4. Arti node-node penting dengan bahasa umum

Nama teknis	Pertanyaan yang dijawab	Penjelasan umum
Parameter	Data apa yang kita punya?	Contoh: arus motor, tegangan keluaran, frekuensi, suhu.
Indicator	Apa arti gabungan data tersebut?	Contoh: suatu nilai dianggap tinggi, rendah, tidak stabil, atau tidak sesuai konteks.
EvidencePattern	Kondisi apa yang terlihat?	Pola beberapa data atau event yang muncul bersama.
ConditionHypothesis	Kemungkinan apa yang perlu diperiksa?	Bukan diagnosis; hanya kandidat pemeriksaan yang didukung oleh evidence.
InspectionCheck	Apa yang harus dicek?	Daftar objek atau kondisi yang perlu diperiksa teknisi.
InspectionStep	Bagaimana cara mengeceknya?	Langkah praktis di lapangan bila langkah tersedia.
DataGap	Data apa yang masih kurang?	Informasi yang perlu dilengkapi agar penilaian lebih kuat.
FaultCode	Fault apa yang diberikan VSD?	Event resmi perangkat yang dapat menghentikan atau mengganggu operasi.
WarningEvent	Peringatan apa yang diberikan VSD?	Peringatan atau informasi kondisi dari perangkat.
MitigationAction	Apa tindakan terkait event?	Tindakan yang terkait dengan fault/warning, tetap dipisahkan dari langkah inspeksi.
Source	Informasi ini berasal dari mana?	Manual, dokumen teknis, atau referensi yang menjadi dasar pengetahuan.

Kenapa pemisahan ini penting?
Karena 'apa yang perlu diperiksa' tidak selalu sama dengan 'tindakan resmi untuk fault'. Dengan memisahkannya, sistem tidak mencampur diagnosis, inspeksi, dan tindakan OEM.

5. Contoh nyata: dari kode VSD sampai tindakan

Misalkan ACQ580 mengirim fault 2310. Pengguna tidak perlu membaca semua istilah teknis terlebih dahulu.

Urutan tampilan	Contoh
Kode	2310
Judul yang mudah dipahami	Arus keluaran terlalu tinggi
Penjelasan	VSD mendeteksi arus keluaran melewati batas internal. Kondisi juga dapat berkaitan dengan gangguan ke tanah atau kehilangan fasa suplai.
Langkah selanjutnya	Periksa beban, motor, dan kabel motor
Detail resmi jika dibutuhkan	cause_official dan action_text dari sumber OEM

Pengguna umum melihat informasi ringkas. Engineer masih dapat membuka detail resmi jika membutuhkan penelusuran lebih dalam. Dengan cara ini, satu database melayani dua kebutuhan tanpa mencampur tingkat bahasa.

Bahasa pengguna vs bahasa teknis

Untuk pengguna	Untuk engineer / audit
user_name	name / official name
user_description	cause_official
MitigationAction.name	action_text
Kalimat ringkas dan familiar	Teks sumber, istilah teknis, provenance

6. Hal yang tidak boleh disalahartikan

ConditionHypothesis bukan diagnosis
Jika graph menyebut suatu kemungkinan, artinya kondisi itu layak diperiksa. Bukan berarti sistem sudah membuktikan penyebab kerusakan.

FaultCode bukan root cause otomatis
Fault adalah fakta dari perangkat. Penyebab sebenarnya dapat memerlukan pemeriksaan motor, kabel, suplai listrik, proses, sensor, atau kondisi mekanis.

Datasheet bukan baseline sehat
Nilai rating dari manual hanya menjelaskan kemampuan atau spesifikasi perangkat. Nilai normal pada aset tertentu harus dibangun dari kondisi operasi aktual.

Jenis nilai	Makna
Datasheet rating	Batas atau spesifikasi dari pabrikan.
Healthy baseline	Perilaku normal aset tertentu ketika sehat.
Live measurement	Nilai yang sedang terbaca saat ini.
Diagnostic threshold	Batas yang dipakai untuk membantu penilaian kondisi.

7. Kondisi database saat ini

Database sudah memiliki baseline kesehatan. Angka ini bukan sekadar jumlah data; angka ini dipakai sebagai patokan agar perubahan di masa depan tidak merusak struktur.

Komponen	Jumlah	Makna
FaultCode	195	Event fault yang sudah terdokumentasi.
WarningEvent	178	Event warning/informasi yang sudah terdokumentasi.
MitigationAction	373	Tindakan yang dimiliki setiap event.
ConditionHypothesis	17	Kemungkinan yang dapat ditelusuri melalui jalur pemeriksaan.
EvidencePattern	17	Pola evidence yang mendukung kemungkinan pemeriksaan.
Bahasa pengguna lengkap	373/373	Semua fault dan warning sudah dapat ditampilkan dalam bahasa pengguna.
Orphan action	0	Tidak ada tindakan yang kehilangan pemilik event.
Multi-owner action	0	Tidak ada tindakan yang dimiliki lebih dari satu event secara tidak sengaja.

Apa arti HOLD?

HOLD berarti bagian tersebut dianggap stabil. Jangan diubah hanya untuk merapikan atau membuat graph terlihat lebih simetris. Perubahan dilakukan jika ada sumber baru, koreksi fakta, produk/firmware baru, atau hasil regression test.

8. Apa yang dapat ditambahkan ke depan?

Fondasi yang ada sudah cukup untuk pengetahuan produk. Pengembangan berikutnya sebaiknya membuat database semakin memahami kondisi aset nyata di lapangan.

Pengembangan	Contoh	Manfaat
Asset nyata	VSD Pompa 03, Motor 500 kW, Pompa Unit 03	Database tahu perangkat mana yang benar-benar terpasang.
Lokasi / Site	Rumah Pompa A, Panel 2	Informasi dapat dipisahkan per lokasi.
Baseline per aset	Arus normal pada 40 Hz	Sistem dapat membandingkan kondisi aset dengan perilaku normalnya sendiri.
Operating Session	Sesi operasi tanggal/jam tertentu	Kejadian dapat dilihat dalam konteks waktu operasi.
Anomaly Event	Arus meningkat tidak biasa	Hasil analisis dapat disimpan sebagai kejadian bermakna.
Maintenance Event	Bearing diganti, panel dibersihkan	Riwayat tindakan lapangan dapat dihubungkan dengan kondisi aset.
Document Chunk	Potongan manual atau laporan	GraphRAG dapat menemukan bagian dokumen yang relevan.
Vector Index	Pencarian berdasarkan kemiripan makna	Pengguna dapat mencari dengan bahasa natural, tidak harus tahu nama node.

Contoh perkembangan dari pengetahuan umum menjadi pengetahuan aset

ACQ580 secara umum	->	**VSD Pompa 03**	->	**Baseline Pompa 03**	->	**Riwayat kejadian Pompa 03**

9. Apa yang tidak perlu dimasukkan ke Neo4j?

Neo4j bukan tempat untuk semua jenis data. Salah satu kesalahan umum adalah memasukkan setiap sampel telemetry ke graph.

Jangan jadikan Neo4j time-series database

Data MQTT setiap detik lebih cocok disimpan di penyimpanan time-series atau data historis. Neo4j cukup menyimpan ringkasan yang bermakna: sesi operasi, anomaly event, baseline, hasil inspeksi, dan maintenance.

MQTT / Telemetry	**->**	**Time-series storage**	**->**	**Analisis**	**->**	**Ringkasan penting**	**->**	**Neo4j**

Dengan pemisahan ini, Neo4j tetap ringan untuk penelusuran hubungan dan tidak berubah menjadi gudang data mentah yang sulit dikelola.

10. Bagaimana Neo4j dioptimalkan?

Optimasi bukan berarti langsung menambah AI. Urutan yang paling sehat adalah menjaga data benar, kemudian membuat query cepat, baru menambah kemampuan pencarian dan AI.

Prioritas	Apa yang dilakukan	Kenapa penting
1. Constraint & identity	Menentukan data apa yang harus unik.	Mencegah node ganda atau identitas salah.
2. Index	Membuat pencarian berdasarkan product, code, ID menjadi cepat.	Mengurangi scan seluruh database.
3. Query terkontrol	Aplikasi memakai Cypher yang jelas dan dibatasi.	Lebih aman dan lebih mudah diaudit.
4. EXPLAIN / PROFILE	Melihat cara Neo4j menjalankan query.	Menemukan query lambat, scan besar, atau pola tidak efisien.
5. Schema health test	Membandingkan graph dengan baseline otomatis.	Perubahan salah dapat diketahui sebelum deployment.
6. Vector search	Ditambahkan ketika semantic search dibutuhkan.	Membantu pencarian bahasa natural tanpa menggantikan graph reasoning.

Identity penting untuk FaultCode / WarningEvent

Kode saja tidak selalu cukup untuk mengenali event. Satu kode dapat muncul pada konteks atau nama resmi yang berbeda. Karena itu identitas yang aman perlu mempertimbangkan produk, kode, dan nama resmi; pada kasus tertentu sumber atau firmware juga perlu dipakai.

Product	+	Code	+	Official Name	=	Identity lebih aman

11. Bagaimana AI dan GraphRAG masuk nanti?

GraphRAG berarti AI tidak hanya mencari teks yang mirip, tetapi juga memanfaatkan hubungan di graph. Contohnya, pencarian semantik menemukan node yang relevan, kemudian Neo4j menelusuri hubungan ke pola, kemungkinan, pemeriksaan, atau sumber.

Pertanyaan user	**->**	**Pencarian semantic**	**->**	**Node kandidat**	**->**	**Graph traversal**	**->**	**Konteks terstruktur**	**->**	**AI menjelaskan**

Vector search mencari kandidat, bukan memberi diagnosis
Embedding membantu menemukan informasi yang mirip secara makna. Keputusan dan hubungan utama tetap mengikuti graph dan aturan yang sudah dikendalikan.

Untuk tahap awal, Neo4j Vector Index dapat dipertimbangkan sebelum menambah database vector terpisah. Database tambahan baru diperlukan jika ada kebutuhan skala atau performa yang benar-benar terbukti.

12. Panduan untuk pengguna aplikasi

Pengguna aplikasi seharusnya tidak perlu melihat struktur database. Yang perlu terlihat adalah informasi yang membantu pekerjaan.

Yang sebaiknya terlihat	Yang sebaiknya disembunyikan dari tampilan utama
Kode event	Internal ID
Judul yang mudah dipahami	entry_type mentah
Penjelasan singkat	audit_*
Langkah selanjutnya	*_status internal
Detail resmi bila dibuka	engineering_detail, matrix_*, formula_or_logic
Sumber bila pengguna membutuhkan validasi	Property teknis yang tidak membantu tindakan

Alur baca yang diharapkan:

Apa yang terlihat?	->	**Apa artinya?**	->	**Apa yang perlu diperiksa?**	->	**Bagaimana mengecek nya?**	->	**Langkah selanjutnya**

Tujuan tampilan

Pengguna harus mendapat jawaban yang membantu tindakan tanpa harus memahami schema Neo4j.

13. Aturan agar database tetap sehat

1. Jangan menambah hubungan hanya agar graph terlihat lengkap.
2. Jangan mengubah teks resmi OEM untuk membuatnya lebih enak dibaca. Buat bahasa pengguna di field terpisah.
3. Jangan menganggap fault sebagai root cause otomatis.
4. Jangan menganggap hypothesis sebagai diagnosis.
5. Jangan memasukkan telemetry mentah dalam jumlah besar tanpa alasan.
6. Lakukan dry-run sebelum SET, MERGE, atau REMOVE.
7. Jalankan schema health / regression test setelah perubahan besar.
8. Catat sumber setiap pengetahuan penting.
9. Pisahkan bahasa pengguna, detail engineering, dan provenance.
10. Tambahkan node baru karena kebutuhan nyata, bukan karena ingin membuat schema simetris.

Baseline yang harus tetap dipantau

Indikator kesehatan	Nilai baseline
FaultCode	195
WarningEvent	178
MitigationAction	373
ConditionHypothesis	17
EvidencePattern	17
Language complete	373
HAS_ACTION relationships	373
Orphan actions	0
Multi-owner actions	0
Forbidden ConditionHypothesis -> REQUIRES_DATA	0
Forbidden InspectionStep <-> MitigationAction	0

14. Roadmap pengembangan yang disarankan

Pengembangan tidak perlu meloncat langsung ke agent AI. Urutan berikut menjaga fondasi tetap sederhana dan mudah diuji.

Tahap	Fokus	Hasil yang diharapkan
1	Automated schema health test	Database dapat memeriksa dirinya sendiri setelah perubahan.
2	Constraint & index audit	Identity lebih aman dan query lebih cepat.
3	Query performance audit	Query utama terukur dengan EXPLAIN/PROFILE.
4	Presentation layer	Aplikasi menampilkan bahasa yang mudah dipahami.
5	Asset & site model	Graph mulai memahami aset nyata dan lokasi.
6	Baseline & history	Sistem memahami perilaku normal dan riwayat aset.
7	Document chunk + vector index	Pencarian dokumen dan semantic search mulai terintegrasi.
8	Advanced GraphRAG	AI menggunakan konteks graph yang sudah stabil.

Urutan berpikir

Database benar -> schema stabil -> query cepat -> retrieval stabil -> tampilan jelas -> baru kemampuan AI diperluas.

15. Glosarium singkat

Istilah	Arti dalam dokumen ini
Graph	Kumpulan informasi beserta hubungan antar-informasi.
Node	Satu objek atau konsep di Neo4j.
Relationship	Hubungan antar-node.
Property	Keterangan yang disimpan pada node atau relationship.
Cypher	Bahasa query untuk membaca atau mengubah data Neo4j.
OEM	Pabrikan asli perangkat. Dalam konteks ini banyak referensi berasal dari ABB.
Provenance	Jejak sumber: informasi berasal dari dokumen atau referensi mana.
Baseline	Gambaran perilaku normal yang dipakai sebagai pembanding.
Vector Index	Indeks yang membantu pencarian berdasarkan kemiripan makna.
Embedding	Representasi numerik suatu teks agar kemiripan makna dapat dihitung.
GraphRAG	Retrieval yang menggabungkan pencarian informasi dengan hubungan di graph sebelum AI menjawab.
Regression Test	Tes untuk memastikan perubahan baru tidak merusak bagian yang sebelumnya sudah benar.
HOLD	Bagian dianggap stabil dan tidak diubah tanpa alasan yang terverifikasi.

Penutup

Neo4j dalam sistem ini bukan sekadar database fault. Ia dibangun sebagai peta pengetahuan yang menghubungkan produk, event, parameter, indikasi, kemungkinan pemeriksaan, tindakan, dan sumber. Nilai utamanya muncul ketika hubungan itu tetap jelas, dapat diaudit, dan dapat diterjemahkan menjadi informasi yang mudah digunakan oleh orang lapangan maupun aplikasi.

Arah pengembangan terbaik bukan memperbanyak node sebanyak mungkin, melainkan menambah konteks yang benar: aset nyata, baseline, riwayat operasi, dokumen, dan kemampuan retrieval. Dengan fondasi yang stabil, AI dapat ditambahkan sebagai lapisan penjelas tanpa mengambil alih peran sumber kebenaran.